

CDS View 課程練習 3：Association vs. JOIN

Lecture

這一課要教什麼

cds01 / cds02 的範例都只碰一張表。現實世界的查詢常常需要跨表取資料——例如查航班時刻表，同時想知道航空公司的中文全名，而不是只有 `carrid` 這種技術代碼。傳統做法是寫一句 `INNER JOIN`；CDS View 提供了另一個工具：**Association**。這一課要搞懂兩者的差異、什麼時候該用哪一個，以及 Association 一個容易被誤解的關鍵行為：**宣告一個 Association，不代表查詢時一定會真的去 JOIN 資料庫。**

這一課會建三個物件，兩兩對照：

1. `ZI_CDS03_FLIGHT_SCHEDULE`：宣告 `_Carrier` Association，但不消費它（只把 Association 本身放進欄位清單，不取用它底下任何欄位）
2. `ZC_CDS03_FLIGHT_WITH_CARRIER`：疊在第一個 View 上面，真正引用 `_Carrier.carname`
3. `ZI_CDS03_FLIGHT_JOIN`：完全不用 Association，直接寫 `INNER JOIN` 達到一樣的結果

語法元素講解

① **Association 宣告**：在 `select from` 之後、欄位清單 `{ }` 之前宣告：

```
association [cardinality] to <目標> as _<別名>
on _<別名>.<欄位> = <來源別名>.<欄位>
```

- **cardinality**（基數）：`[1..1]` 代表目標剛好一筆（例如每個航班一定屬於一家航空公司）；`[0..1]` 代表目標零或一筆（可能查不到）；`[0..*]` / `[1..*]` 代表目標可能多筆。這一課的 `_Carrier` 用 `[1..1]`，因為 `SPFLI` 的每一筆航班排程，一定對應到 `SCARR` 剛好一筆航空公司主檔。
- **_ 前綴**：Association 別名習慣上以底線開頭（`_Carrier`），這是 SAP 的命名慣例，用來跟一般欄位別名做視覺區分，不是語法強制要求，但幾乎所有官方範例都這樣寫，這門課也照這個慣例。
- **on 條件參照來源別名**：舊式 `define view`（這系統唯一支援的版本）裡，`on` 條件左右兩邊要嘛寫 Association 的別名（`_Carrier.carrid`），要嘛寫來源表自己的別名——這代表來源表通常需要明確給一個 `as <別名>`，這一課把 `spfli` 命名成 `as Schedule`，`on` 條件才能寫成 `_Carrier.carrid = Schedule.carrid`。

② 欄位清單裡引用 **Association** 的兩種方式：

```
{
  key Schedule.carrid,
  ...
  _Carrier
}
```

" 方式一：只公開這個 Association 本身，不取用底下欄位

```
{
  key carrid,
  ...
}
```

```
_Carrier.carrname as CarrierName    " 方式二：直接取用 Association 目標的某個欄位
}
```

這兩種寫法的差異，正是這一課的核心觀念——方式一只是「公開」這個關聯，告訴消費這個 View 的下一層「你可以透過 `_Carrier` 走到航空公司資料」，但這一層自己不會因此去查 `SCARR`；方式二直接引用了 `_Carrier` 底下的欄位，這時候查詢引擎才會真的產生一個 JOIN 把 `SCARR` 接上來。

⚠ 最重要的觀念：Association 只有「被引用」才會轉譯成 SQL JOIN

這一課刻意設計了三層對照，用實測結果直接證明這件事：

第一層 `ZI_CDS03_FLIGHT_SCHEDULE`——宣告了 `_Carrier`，但欄位清單裡只寫 `_Carrier`（方式一，只公開不取用）：

```
define view ZI_CDS03_FLIGHT_SCHEDULE
as select from spfli as Schedule
association [1..1] to scarr as _Carrier
on _Carrier.carrid = Schedule.carrid
{
key Schedule.carrid,
key Schedule.connid,
  Schedule.cityfrom,
  Schedule.cityto,
  Schedule.deptime,
  Schedule.arrrtime,

  _Carrier
}
```

直接查這個 View（見下方驗證結果第 1 段），完全沒有航空公司名稱可以取——因為這一層根本沒有引用 `_Carrier` 底下任何欄位，資料庫層面不會發生 JOIN。

第二層 `ZC_CDS03_FLIGHT_WITH_CARRIER`——疊在第一層之上，這次真正引用 `_Carrier.carrname`：

```
define view ZC_CDS03_FLIGHT_WITH_CARRIER
as select from ZI_CDS03_FLIGHT_SCHEDULE
{
key carrid,
key connid,
  cityfrom,
  cityto,
  deptime,
  arrtime,

  _Carrier.carrname as CarrierName
}
```

這一層因為真正取用了 `_Carrier.carrname`，查詢時才會產生對 `SCARR` 的 JOIN，`CarrierName` 欄位才查得到值（見驗證結果第 2 段，正確顯示 `Lufthansa`）。

這代表什麼？Association 是一種延遲宣告（lazy declaration）——你可以在 Interface View 這一層先把「這裡可以走到哪些關聯資料」都宣告好，但真正要不要付出 JOIN 的成本，交給消費端（下一層 View，或最終呼叫的 ABAP 程式）決定。如果消費端從頭到尾都沒有取用某個 Association 底下的欄位，資料庫層根本不會執行那個 JOIN——這是直接寫死 JOIN 完全做不到的彈性。

對照組：直接寫 JOIN

```
define view ZI_CDS03_FLIGHT_JOIN
  as select from spfli as Schedule
  inner join    scarr as Carrier
    on Carrier.carrid = Schedule.carrid
{
  key Schedule.carrid,
  key Schedule.connid,
    Schedule.cityfrom,
    Schedule.cityto,
    Schedule.deptime,
    Schedule.arrrtime,

    Carrier.carrname    as CarrierName
}
```

這個版本不管消費端有沒有用到 `CarrierName`，每次查詢都一定會執行 JOIN——因為 JOIN 是寫死在這一層 View 的定義裡，不是延遲決定的。驗證結果第 3 段跟第 2 段（Association 路徑）的 `CarrierName` 完全一致，證明兩種寫法查出來的資料是一樣的，差別純粹在「JOIN 什麼時候發生、由誰決定」。

什麼時候該用 Association，什麼時候該用 JOIN？

情境	建議
這份關聯資料不是每次查詢都需要（例如 Interface View 給多個不同用途的消費端共用，有些消費端要航空公司名稱，有些不要）	用 Association ——把彈性留給消費端，不需要的人不用付出 JOIN 成本
這份關聯資料這一層一定會用到（例如就是要做一個「航班+航空公司」的報表專用 View，任何呼叫這個 View 的人都一定需要航空公司名稱）	用 JOIN 更直接，語意上也更清楚「這兩張表本來就是綁在一起的」
要在下一層 View 才決定要不要引用某個關聯	只能用 Association——JOIN 沒有這種「宣告但不執行」的能力

Eclipse ADT 建立 CDS View：Step by Step

以 `ZI_CDS03_FLIGHT_SCHEDULE` 為例（另外兩個物件重複同樣步驟）：

- 1. 對著 `$TMP` 套件右鍵 → **New** → **Other ABAP Repository Object** → 篩選 `Data Definition` → **Next**

2. Name : `ZI_CDS03_FLIGHT_SCHEDULE` · Description : `CDS03 Association vs JOIN` · Package : `$TMP`
→ **Next** → Transport 畫面直接 **Finish**
3. Templates 畫面選 **Define View** (**obsolete as of AS ABAP 7.57**)
4. Reference Object 選 `SPFLI`
5. 改成本課範例的內容——注意 `as select from spfli as Schedule` 這裡要自己手動加上 `as Schedule` · 精靈帶出的骨架預設不會有來源別名
6. **Ctrl+S** 存檔 → **Activate** (**Ctrl+F3**)
7. `ZC_CDS03_FLIGHT_WITH_CARRIER` 建立時 · Reference Object 這一步不用選任何表 (因為它是查另一個 CDS View · 不是查底層表) · 直接手動打完整內容即可
8. 三個物件都建好、啟用成功後 · 對 `ZC_CDS03_FLIGHT_WITH_CARRIER` 跟 `ZI_CDS03_FLIGHT_JOIN` 分別用 **Data Preview** 確認 `CarrierName` 欄位查得到資料；對 `ZI_CDS03_FLIGHT_SCHEDULE` 用 Data Preview 應該只看得到排程欄位 · 看不到任何航空公司名稱相關的東西

這一課學到的東西 · 接下來會怎麼用

- `cds04` : Parameters — 這一課的 `on` 條件是寫死的比較 · `cds04` 會教怎麼讓查詢條件在執行期才決定
- `cds05` : 分層設計 — 這一課的兩層架構 (Interface View 宣告 Association · Composite View 消費它) 正是 `cds05` 要正式定名、深入講解的設計模式
- `cds09` (進階篇) : Extend View — 另一種在不修改原始碼的前提下幫既有 View 加欄位 / 加 Association 的機制

Eclipse ADT Step by Step (重點回顧)

1. `ZI_CDS03_FLIGHT_SCHEDULE` : 以 `SPFLI` 為來源建立 · 宣告 `_Carrier` Association · 欄位清單只公開 Association 本身 (不取用底下欄位)
2. `ZC_CDS03_FLIGHT_WITH_CARRIER` : 以 `ZI_CDS03_FLIGHT_SCHEDULE` 為來源 · 真正引用 `_Carrier.carrname`
3. `ZI_CDS03_FLIGHT_JOIN` : 以 `SPFLI` 為來源 · 直接寫 `INNER JOIN SCARR` · 效果對照組
4. 三個物件都用 Data Preview 驗證：第一個查不到航空公司名稱 · 後兩個查得到且結果一致

學習目標

- 能寫出 Association 宣告語法 (`association [cardinality] to <目標> as _<別名> on ...`) · 並選對正確的 cardinality
- 能講出「宣告 Association 只是公開一個可能的關聯 · 只有被引用 (欄位清單裡真正取用 `_別名.欄位`) 才會轉譯成 SQL JOIN」這個核心觀念 · 並能舉出這一課的實測證據 (`ZI_CDS03_FLIGHT_SCHEDULE` 查不到航空公司名稱 · `ZC_CDS03_FLIGHT_WITH_CARRIER` 查得到)
- 能寫出直接 `INNER JOIN` 的 CDS View · 並理解它跟 Association 的本質差異 (JOIN 是寫死執行的 · Association 是延遲決定的)
- 能判斷什麼情境該用 Association · 什麼情境該直接寫 JOIN
- 知道 Association 的 `on` 條件通常需要來源表有明確別名 (`as <別名>`) 才能正確參照

物件清單

物件	名稱	型別
CDS Interface View (宣告 Association · 不消費)	<code>ZI_CDS03_FLIGHT_SCHEDULE</code>	DDLS/DF

物件	名稱	型別
CDS Consumption View (消費 Association)	ZC_CDS03_FLIGHT_WITH_CARRIER	DDLS/DF
CDS Interface View (直接 JOIN 對照組)	ZI_CDS03_FLIGHT_JOIN	DDLS/DF
驗證程式	ZR_CDS03_DEMO	PROG/P

全部物件都在 **\$TMP** 套件，已建立並啟用（讀取 active 版本內容與寫入內容逐字相符）。

動手練習物件（你自己動手建，名稱自訂，不算在上面正式的課程物件清單裡）：

物件	建議名稱	型別	對應練習
CDS View (SFLIGHT 關聯到 SPFLI)	ZI_CDS03_FLIGHT_DETAIL (自訂)	DDLS/DF	動手練習

動手練習

輪到你了：**SFLIGHT**（航班訂位明細）跟 **SPFLI**（航班排程）之間也有天然的關聯（**SFLIGHT** 的 **carrid** + **connid** 對應到 **SPFLI** 的排程主檔）。建一個新的 CDS View（物件名稱、套件 **\$TMP**，命名自訂，例如 **ZI_CDS03_FLIGHT_DETAIL**），要求：

1. 以 **SFLIGHT** 為來源（記得給它一個明確別名，例如 **as Flight**）
2. 宣告一個 Association（例如 **_Schedule**）指到 **SPFLI**，**on** 條件用 **carrid** + **connid** 兩個欄位比對（cardinality 建議用 **[1..1]**，因為每筆訂位明細對應到剛好一筆排程）
3. 欄位清單**直接引用** **_Schedule.cityfrom** 跟 **_Schedule.cityto**（不要只公開 Association 本身，這一題要親自體驗「引用了就會真的 JOIN」）
4. 用 Data Preview 確認查得到出發城市 / 抵達城市

如果你想額外挑戰一下：把第 3 步改成只公開 **_Schedule** 本身、不引用底下任何欄位，重新啟用後再看一次 Data Preview——確認這一版真的看不到城市資訊，親自驗證「不引用就不會 JOIN」這件事，而不是只相信講義說的。

建好、啟用成功後跟我說一聲，我會幫你核對語法。

驗證方式

ZR_CDS03_DEMO 透過 **programrun** 無頭驗證，依序查三個 View（篩選 **carrid** = 'LH'），並比對 Association 路徑跟直接 JOIN 路徑查出來的航空公司名稱是否一致：

```
=== 1. ZI_CDS03_FLIGHT_SCHEDULE (association declared, not consumed) ===
LH  0400 FRANKFURT          -> NEW YORK
(no carrier name exposed at this layer)
LH  0401 NEW YORK           -> FRANKFURT
(no carrier name exposed at this layer)
LH  0402 FRANKFURT          -> NEW YORK
(no carrier name exposed at this layer)
=== 2. ZC_CDS03_FLIGHT_WITH_CARRIER (consumes _Carrier association) ===
LH  0400 FRANKFURT          -> NEW YORK          | Lufthansa
LH  0401 NEW YORK           -> FRANKFURT         | Lufthansa
```

```

LH  0402 FRANKFURT      -> NEW YORK      | Lufthansa
=== 3. ZI_CDS03_FLIGHT_JOIN (direct INNER JOIN) ===
LH  0400 FRANKFURT      -> NEW YORK      | Lufthansa
LH  0401 NEW YORK       -> FRANKFURT     | Lufthansa
LH  0402 FRANKFURT      -> NEW YORK      | Lufthansa
=== 4. Carrier names match between Association-path and direct JOIN? ===
MATCH: carrier names identical via both paths, row count      3

```

第 1 段完全沒有航空公司名稱（因為沒有引用 `_Carrier` 底下的欄位），第 2、3 段都正確查到 `Lufthansa`，第 4 段確認兩條路徑（Association vs. 直接 JOIN）查出來的資料完全一致——證實這一課的核心觀念不是紙上談兵，是這個系統上真實可驗證的行為。

動手練習的驗證方式：Eclipse 啟用成功 + 用 Data Preview 看查詢結果，貼程式碼給我核對語法。

思考題

1. 如果把 `ZI_CDS03_FLIGHT_SCHEDULE` 的 `_Carrier` cardinality 從 `[1..1]` 改成 `[0..1]`，語意上代表什麼差異？在 `SPFLI` / `SCARR` 這組資料的實際情境下，哪一個 cardinality 比較貼近事實？
2. 這一課示範的是「兩層 View：Interface View 宣告不消費、Composite View 消費」，如果你今天只需要一層（一個 View 從頭到尾都會用到航空公司名稱），要不要多繞這兩層？直接在同一個 View 裡宣告 Association 並馬上引用底下欄位可不可以？
3. `ZI_CDS03_FLIGHT_JOIN` 每次查詢都會執行 JOIN，就算呼叫端根本沒用到 `CarrierName` 欄位（例如只 `SELECT carrid, connid FROM zi_cds03_flight_join`）。你覺得資料庫在這種情境下，還是會不會真的去執行 JOIN？如果會，這對效能有什麼影響？（提示：這是 Association 相對 JOIN 的核心優勢，回顧「什麼時候該用哪一個」那一節）
4. Association 的 `on` 條件目前都是寫死的等值比對（`_Carrier.carrid = Schedule.carrid`）。如果你想要「只關聯到某個特定狀態的資料」（例如只關聯到「啟用中」的航空公司），`on` 條件可以加更多比較條件嗎？試著查一下 CDS Association 語法文件，或直接動手試試看。

答案

見 `zi_cds03_flight_schedule.ddls.abap`、`zc_cds03_flight_with_carrier.ddls.abap`、`zi_cds03_flight_join.ddls.abap`、`zr_cds03_demo.prog.abap`。SAP 端物件：`ZI_CDS03_FLIGHT_SCHEDULE` / `ZC_CDS03_FLIGHT_WITH_CARRIER` / `ZI_CDS03_FLIGHT_JOIN`（CDS View）、`ZR_CDS03_DEMO`（驗證程式）。動手練習（`SFLIGHT` 關聯 `SPFLI`）由你在 Eclipse 動手建立，沒有固定答案快照——建好後跟我核對即可。